Fast Fourier Transform (FFT) Algorithm

Morten HJ

Old notes from Comp Phys on FFT

Motivation

- The Fourier Transform is a fundamental tool in signal processing, physics, engineering, and applied mathematics
- Direct computation of the Discrete Fourier Transform (DFT) is computationally expensive: $O(N^2)$
- The Fast Fourier Transform (FFT) reduces this to $O(N \log N)$
- Applications include:
  - Signal processing (audio, image, video)
  - Solving partial differential equations
  - Polynomial multiplication
  - Data compression
  - Spectral analysis

Historical Context

- First discovered by Gauss in 1805 (unpublished) for calculating asteroid orbits
- Rediscovered by Cooley and Tukey in 1965
- Popularized with the advent of digital computers
- One of the most important numerical algorithms of the 20th century

Discrete Fourier Transform (DFT)

The DFT of a sequence $x[n]$ of length N is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1$$

where:

- $x[n]$ is the input sequence (time domain)
- $X[k]$ is the output sequence (frequency domain)
- N is the number of samples
- $j = \sqrt{-1}$ is the imaginary unit

Inverse DFT

The inverse transform is given by:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi kn/N}, \quad n = 0, 1, \ldots, N-1$$

Key Properties

- Linearity: $ax_1[n] + bx_2[n] \leftrightarrow aX_1[k] + bX_2[k]$
- Periodicity: $X[k + N] = X[k]$
- Symmetry: For real $x[n]$, $X[N - k] = X^*[k]$
- Convolution: Multiplication in one domain equals convolution in the other

FFT Basic Idea

- Exploits symmetry and periodicity of the complex exponential
- Decimates the DFT computation into smaller DFTs
- Most common approach: Cooley-Tukey algorithm (radix-2)
- Requires N to be a power of 2 (can be generalized)

Divide and Conquer Approach

Split the DFT sum into even and odd terms:

$$
\begin{aligned}
X[k] &= \sum_{n=0}^{N-1} x[n] W_N^{kn} \\
&= \sum_{m=0}^{N/2-1} x[2m] W_N^{k(2m)} + \sum_{m=0}^{N/2-1} x[2m+1] W_N^{k(2m+1)} \\
&= \sum_{m=0}^{N/2-1} x_{\text{even}}[m] W_{N/2}^{km} + W_N^k \sum_{m=0}^{N/2-1} x_{\text{odd}}[m] W_{N/2}^{km} \\
&= E[k] + W_N^k O[k]
\end{aligned}
$$

where $W_N = e^{-j2\pi/N}$ is the twiddle factor.

Recursive Structure

- The DFT of size N is decomposed into:
  - Two DFTs of size $N/2$ (even and odd terms)
  - $O(N)$ complex multiplications and additions
- Applying this recursively gives $O(N \log N)$ complexity

Butterfly Operation

The basic computational unit of the FFT:

Mathematically:

$$X[m] = E[m] + W_N^m O[m]$$
$$X[m + N/2] = E[m] - W_N^m O[m]$$

Pseudocode for Radix-2 FFT

Algorithm 1 Recursive Radix-2 FFT

Require: N is a power of 2, x is input array of size N

1: **if** $N == 1$ **then**
2: **return** x
3: **end if**
4: $x_{\text{even}} \leftarrow [x[0], x[2], \ldots, x[N-2]]$
5: $x_{\text{odd}} \leftarrow [x[1], x[3], \ldots, x[N-1]]$
6: $E \leftarrow \text{FFT}(x_{\text{even}})$
7: $O \leftarrow \text{FFT}(x_{\text{odd}})$
8: $W_N \leftarrow e^{-2\pi j/N}$
9: $W \leftarrow 1$
10: **for** $k = 0$ to $N/2 - 1$ **do**
11: $X[k] \leftarrow E[k] + W \cdot O[k]$
12: $X[k + N/2] \leftarrow E[k] - W \cdot O[k]$
13: $W \leftarrow W \cdot W_N$
14: **end for**

Python Implementation

```
1   import numpy as np
2
3   def fft(x):
4   N = len(x)
5   if N == 1:
6   return x
7   even = fft(x[::2])
8   odd = fft(x[1::2])
9   T = [np.exp(-2j * np.pi * k / N) * odd[k]
10    for k in range(N // 2)]
11  return [even[k] + T[k] for k in range(N // 2)] + \
12  [even[k] - T[k] for k in range(N // 2)]
13
```

Iterative Implementation

- Recursive implementation has overhead
- More efficient: iterative version with bit-reversal permutation
- Three main steps:
  1. Bit-reverse the input array
  2. Perform butterfly operations at each level
  3. Combine results

Computational Complexity

- Let $N = 2^m$
- At each level k ($k = 1$ to m):
  - 2^{k-1} butterfly operations
  - Each butterfly has 1 complex multiplication and 2 additions
  - Total operations per level: $N/2$ butterflies $\times$ 3 operations
- Total levels: $\log_2 N$
- Total operations: $\frac{3}{2} N \log_2 N$
- Thus, complexity is $O(N \log N)$ vs. $O(N^2)$ for DFT

Comparison with Direct Computation

For $N = 1024$:

- Direct DFT: $N^2 = 1,048,576$ complex multiplications
- FFT: $\frac{N}{2} \log_2 N = 5120$ complex multiplications
- Speedup factor: 205 times

N	DFT Operations	FFT Operations
32	1,024	80
64	4,096	192
128	16,384	448
256	65,536	1,024
512	262,144	2,304
1024	1,048,576	5,120

Different FFT Algorithms

- **Radix-2**: Most common, requires N power of 2
- **Radix-4**: More efficient for certain architectures
- **Mixed-radix**: For arbitrary N (prime factor algorithm)
- **Bluestein's algorithm**: For arbitrary N (Chirp-Z transform)
- **Winograd FFT**: Minimizes multiplications

Multidimensional FFT

- Separable transform: Apply 1D FFT along each dimension
- For 2D image of size $M \times N$:

$$X[k,l] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x[m,n] e^{-j2\pi(km/M + ln/N)}$$

- Computed as:
  1. Apply 1D FFT to each row ($M \times O(N \log N)$)
  2. Apply 1D FFT to each column ($N \times O(M \log M)$)
- Total complexity: $O(MN \log MN)$

Practical Applications

- **Signal Processing**:
  - Filtering (convolution via multiplication in frequency domain)
  - Spectral analysis
  - Audio compression (MP3, AAC)
- **Image Processing**:
  - JPEG compression (2D FFT on 8x8 blocks)
  - Filtering and enhancement
  - Feature extraction
- **Numerical Analysis**:
  - Solving PDEs (spectral methods)
  - Fast polynomial multiplication
- **Other Fields**:
  - Quantum computing (QFT)
  - Astronomy (interferometry)

Example: Polynomial Multiplication

- Multiply two polynomials $A(x)$ and $B(x)$ of degree $n-1$
- Naive approach: $O(n^2)$ operations
- Using FFT:
  1. Evaluate A and B at $2n$ points using FFT ($O(n \log n)$)
  2. Pointwise multiply the results ($O(n)$)
  3. Interpolate to get coefficients using inverse FFT ($O(n \log n)$)
- Total complexity: $O(n \log n)$

Summary

- FFT is an efficient algorithm to compute the DFT
- Reduces complexity from $O(N^2)$ to $O(N \log N)$
- Based on divide-and-conquer strategy
- Numerous applications across science and engineering
- Modern implementations (FFTW) are highly optimized

Further Reading

- Cooley, J. W.; Tukey, J. W. (1965). "An algorithm for the machine calculation of complex Fourier series". Math. Comput. 19: 297-301.
- Brigham, E. O. (1988). *The Fast Fourier Transform and Its Applications*. Prentice-Hall.
- Van Loan, C. (1992). *Computational Frameworks for the Fast Fourier Transform*. SIAM.
- FFTW: http://www.fftw.org/
- Numerical Recipes in C: The Art of Scientific Computing